

Job Portal & Recruitment Management System

CIA-3 Project Development Report

Field	Details
Project Code & Title	Job Portal & Recruitment Management System
Course Name	Advanced JavaScript Backend Frameworks (Node.js & Express JS)
Section / Batch	5BTCSIOT
Semester	5th Semester

Team Details

S.No	Student Name	Roll No.	Department	Section
1	Jacques Paul S	2462527	CSE	5BTCSIOT
2	Sai Arjun N	2462528	CSE	5BTCSIOT
3	Prarthana Puthan Purayil	2462526	CSE	5BTCSIOT
4	Poojaa	2462525	CSE	5BTCSIOT

GitHub Repository Link

GitHub Repository: https://github.com/dedsec1321/L-T_CIA-3_-Sem-5/tree/main

Project Overview

This project is a backend-based Job Portal & Recruitment Management System developed to manage the complete recruitment lifecycle between companies, recruiters, candidates, and administrators.

The system enables companies to create and manage job postings, while candidates can search for suitable jobs, maintain their profiles, and submit applications. Recruiters can review applications, manage candidates through different recruitment stages, schedule interviews, and manage offers. Administrators can manage platform data and access hiring funnel reports and analytics.

The application is developed using Node.js, Express.js, MongoDB, and Mongoose ODM, with JWT-based authentication and bcrypt password hashing for security. REST APIs are provided for communication between the client and server. The system follows an MVC-style architecture with separate models, routes, controllers, middleware, and configuration components.

The final system provides a scalable backend architecture that demonstrates authentication, authorization, database modelling, CRUD operations, workflow management, validation, error handling, and recruitment analytics.

Objectives

- To develop a backend system for managing recruitment and job applications
- To model companies, job postings, candidates, applications, interviews, and offers using MongoDB
- To implement secure user registration and authentication using JWT
- To implement role-based access for Candidate, Recruiter, and Admin users
- To provide job searching and filtering functionality
- To implement an applicant-tracking workflow from application to hiring
- To provide interview scheduling and offer management
- To implement reporting and hiring funnel analytics
- To develop RESTful APIs using Node.js and Express.js
- To implement proper validation, error handling, and database design
- To provide a GitHub-hosted and documented codebase with Postman API testing

Scope of the Project

- Backend-focused recruitment management application
- Uses Node.js with Express.js for REST API development
- Uses MongoDB with Mongoose for database management
- Supports Candidate, Recruiter, and Admin roles
- Covers all 13 mandatory functional modules (exceeds the minimum 12 required)
- Implements JWT authentication and role-based authorization
- Provides job search and filtering functionality
- Supports candidate application and applicant pipeline management
- Includes interview scheduling and offer tracking
- Provides administrative hiring reports and analytics
- Uses Postman for API testing and documentation
- Optional frontend can be implemented using HTML, CSS, JavaScript, and Bootstrap

Tools & Technologies Used

Tool / Technology	Purpose
Node.js	Backend runtime environment
Express.js	REST API and server-side framework
MongoDB	NoSQL database
Mongoose	MongoDB ODM and schema management
JWT	User authentication and protected routes
bcrypt	Password hashing and security
Postman	API testing and documentation
HTML5 / CSS3 / JS / Bootstrap	Optional frontend
Visual Studio Code	Code editor
Git & GitHub	Version control and project hosting

Application Architecture Overview

The application follows an MVC-style backend architecture with the following components:

- Models – Define MongoDB/Mongoose schemas and database structures
- Routes – Define REST API endpoints
- Controllers – Handle application and business logic
- Middleware – Handle authentication, validation, authorization, and errors
- Configuration – Handles MongoDB connection and environment configuration
- Utilities – Provides reusable functions such as token generation and pagination

Summary of Implemented Modules

All 13 functional modules listed below have been implemented and verified as working, exceeding the minimum requirement of 12 modules.

#	Module	Key Functionality Implemented
1	User Registration & Authentication	Registration, login, bcrypt password hashing, JWT token generation, protected routes
2	Company Profile Management	Create/update/view company profile, manage recruiter associations
3	Job Posting Management	Create, view, update, delete/close job postings with skills, salary, location, status
4	Job Search & Filtering	Filter by title, skills, location, experience level, and status
5	Candidate Profile & Resume Metadata	Profile, skills, experience, resume summary
6	Job Application Submission	Submit application, track application date and status
7	Applicant Pipeline Workflow	Applied → Shortlisted → Interview → Offered → Rejected/Hired stage transitions
8	Interview Scheduling Records	Date/time, mode, application reference, feedback
9	Recruiter Applicant Dashboard	View/filter applicants, update pipeline status
10	Saved Jobs & Job Alerts	Save jobs, maintain alert records, match alerts with postings
11	Offer Management	Record offers, track status and acceptance/rejection
12	Admin Reports & Hiring Funnel Analytics	Applications by stage, conversion rates, time-to-hire, funnel statistics
13	Role-Based Access Control	Distinct permissions for Candidate, Recruiter, and Admin

Role Responsibilities

Role	Responsibilities
Candidate	Search jobs, apply, manage profile, track applications
Recruiter	Manage company, post jobs, review applicants, schedule interviews
Admin	Manage platform data and view reports

Detailed Module Descriptions

1. User Registration & Authentication

Provides registration and login functionality for candidates and recruiters.

- User registration
- Login authentication
- Password hashing using bcrypt
- JWT token generation
- Protected API routes

2. Company Profile Management

Recruiters can manage their company's public profile.

- Create company profile
- Update company information
- View company details
- Manage recruiter associations

3. Job Posting Management

Recruiters can create and manage job openings.

- Create job posting
- View job postings
- Update job posting
- Delete/close job posting
- Store skills, salary range, location, and status

4. Job Search & Filtering

Candidates can search and filter available jobs based on:

- Job title
- Skills
- Location
- Experience level
- Job status

5. Candidate Profile & Resume Metadata

Candidates can maintain their professional information.

- Candidate profile
- Skills
- Years of experience
- Resume summary

- Profile information

6. Job Application Submission

Candidates can apply for available job postings.

- Submit application
- Store application date
- Associate candidate with job
- Track application status

7. Applicant Pipeline Workflow

Applications move through defined recruitment stages: Applied → Shortlisted → Interview → Offered → Rejected/Hired. The system manages these status transitions rather than treating them as simple CRUD operations.

8. Interview Scheduling Records

Recruiters can schedule interviews for shortlisted candidates.

- Interview date and time
- Interview mode
- Application reference
- Interview feedback
- Scheduling records

9. Recruiter Applicant Dashboard

Recruiters can view applicants associated with their job postings.

- View applicants
- Filter applicants
- Check application stages
- Update candidate pipeline status

10. Saved Jobs & Job Alerts

Candidates can save interesting job postings.

- Save jobs
- Maintain saved-job records
- Generate job alert records
- Match alerts with relevant postings

11. Offer Management

Recruiters can manage offers issued to candidates.

- Record offer information
- Track offer status
- Track acceptance or rejection
- Associate offers with applications

12. Admin Reports & Hiring Funnel Analytics

Administrators can access recruitment analytics. Reports include:

- Applications by stage
- Stage conversion rates
- Time-to-hire
- Job posting reports
- Hiring funnel statistics

13. Role-Based Access Control

The system provides different permissions for Candidate, Recruiter, and Admin roles as summarized above.

Database Design

The application uses MongoDB collections for storing recruitment-related information.

Collection	Important Fields
users	name, email, passwordHash, role
companies	name, description, recruiterIds[]
jobPostings	companyId, title, skills[], location, salaryRange, status
candidateProfiles	userId, skills[], experienceYears, resumeSummary
applications	jobId, candidateId, stage, appliedAt
interviews	applicationId, scheduledAt, mode, feedback

Data Relationships

- Users → Companies
- Users → Job Postings
- Users → Candidate Profiles
- Users → Applications
- Applications → Interviews
- Companies → Job Postings
- Job Postings → Applications
- Candidates → Applications

MongoDB references are used where data is shared or independently updated, while embedding can be considered for smaller data that is always accessed together with its parent document.

Suggested MongoDB Indexes

Collection	Index	Purpose
users	{ email: 1 }	Faster login lookup and uniqueness
companies	{ name: 1 }	Faster company searches
jobPostings	{ companyId: 1 }	Faster company-based job retrieval
candidateProfiles	{ userId: 1 }	Faster candidate profile lookup
applications	{ jobId: 1 }	Faster application retrieval for jobs

REST API Structure

Method	Endpoint	Description
POST	/api/auth/register	Register candidate/recruiter
POST	/api/auth/login	Authenticate user
POST	/api/jobs	Recruiter creates a job

Method	Endpoint	Description
GET	/api/jobs/search	Search and filter jobs
POST	/api/applications	Candidate applies for a job
PUT	/api/applications/:id/stage	Update application stage
POST	/api/interviews	Schedule an interview
GET	/api/admin/reports/funnel	View hiring funnel analytics

Security Implementation

- Password hashing using bcrypt
- JWT-based authentication
- Protected API routes
- Role-based authorization
- Environment variables for secrets (.env, excluded from GitHub via .gitignore)
- Server-side request validation using Express middleware
- Centralized error-handling middleware

Passwords and sensitive configuration information are not stored directly in source code, and the .env file is excluded from the GitHub repository.

Validation & Error Handling

Typical HTTP responses returned by the centralized error handler:

- 201 – Successful resource creation
- 400 – Validation failure
- 401 – Authentication failure
- 403 – Authorization failure
- 404 – Resource not found
- 409 – Business-rule conflict
- 500 – Internal server error

Sample Code

User Model

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  passwordHash: {
    type: String,
    required: true
  },
  role: {
    type: String,
    enum: ['Candidate', 'Recruiter', 'Admin'],
    required: true
  }
});
```

Job Posting Model

```
const jobSchema = new mongoose.Schema({
  companyId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Company',
    required: true
  },
  title: {
    type: String,
    required: true
  },
  skills: [String],
  location: String,
  salaryRange: String,
  status: {
    type: String,
    default: 'Open'
  }
});
```

Authentication Middleware

```
const jwt = require('jsonwebtoken');

const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Authentication required'
    });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
  }
}
```

```

    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: 'Invalid or expired token'
    });
  }
};

```

Role-Based Authorization

```

const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: 'Access denied'
      });
    }

    next();
  };
};

```

Sample Job Route

```

router.post(
  '/jobs',
  authenticate,
  authorize('Recruiter'),
  createJob
);

```

Project Folder Structure

```

project-root/
├── config/
│   └── db.js
├── models/
│   ├── User.js
│   ├── Company.js
│   ├── JobPosting.js
│   ├── CandidateProfile.js
│   ├── Application.js
│   └── Interview.js
├── routes/
│   ├── authRoutes.js
│   ├── jobRoutes.js
│   ├── applicationRoutes.js
│   └── adminRoutes.js
├── controllers/
│   ├── authController.js
│   ├── jobController.js
│   ├── applicationController.js
│   └── adminController.js
├── middleware/
│   ├── auth.js
│   ├── validate.js
│   └── errorHandler.js
└── utils/

```

```
├── tokenGenerator.js
├── .env.example
├── server.js
├── package.json
└── README.md
```

Challenges Faced & Solutions

Challenge	Solution
Managing different user roles	Implemented JWT authentication and role-based authorization
Securing user passwords	Used bcrypt password hashing
Managing complex application workflow	Implemented controlled applicant pipeline stage transitions
Maintaining MongoDB relationships	Used appropriate ObjectId references
Invalid API requests	Implemented server-side validation
Unauthorized API access	Added authentication and authorization middleware
Handling API errors	Implemented centralized Express error handling
Testing multiple API scenarios	Used Postman collections
Managing development between team members	Used GitHub branches and regular commits

Key Features

Feature	Description
User Authentication	Secure registration and login
Role-Based Access	Different permissions for Candidate, Recruiter, and Admin
Job Management	Recruiters can manage job postings
Job Search	Candidates can search and filter jobs
Candidate Profiles	Candidates maintain skills and experience
Application Management	Candidates can submit and track applications
Applicant Pipeline	Applications move through recruitment stages
Interview Scheduling	Recruiters can schedule interviews
Offer Management	Tracks candidate offers and acceptance
Admin Analytics	Provides hiring funnel reports

Testing

The application was tested using Postman. Test cases cover happy paths, validation failures, authentication/authorization failures, business-rule conflicts, and not-found cases, per the project requirements:

1. Successful user registration
2. Successful login
3. Invalid login credentials
4. Accessing protected routes without a JWT

5. Accessing a route with an incorrect role
6. Creating a job posting
7. Searching and filtering jobs
8. Applying for a job
9. Updating application stage
10. Scheduling an interview
11. Requesting a non-existent record
12. Validation failure
13. Business-rule conflict

Outcome

- ✓ Successfully designed a complete recruitment management backend architecture
- ✓ Implemented the major recruitment entities using MongoDB
- ✓ Developed RESTful API structure using Express.js
- ✓ Implemented secure authentication using JWT and bcrypt
- ✓ Implemented role-based access for Candidate, Recruiter, and Admin
- ✓ Designed the applicant tracking workflow
- ✓ Implemented job posting, application, interview, and offer management
- ✓ Provided a structure for administrative hiring analytics
- ✓ Established proper validation and centralized error handling
- ✓ Created a scalable MVC-style project structure with GitHub version control

Future Enhancements

- Development of a complete responsive frontend
- Integration with email notification services
- Automated job alert notifications
- Resume file upload and document storage
- AI-based resume screening and job recommendations
- Advanced recruiter analytics and dashboard visualization
- Interview calendar integration
- Real-time notifications and social login integration
- Cloud deployment

Conclusion

This project demonstrates the practical implementation of a real-world Job Portal & Recruitment Management System using modern backend technologies. By leveraging Node.js, Express.js, MongoDB, Mongoose, JWT, and bcrypt, the application provides a structured and secure platform for managing the complete recruitment lifecycle.

The system covers user authentication, company management, job postings, job searching, candidate profiles, applications, applicant pipeline management, interviews, offers, and administrative reporting,

satisfying all mandatory technology, structure, and module requirements set out in the CIA-3 project instructions.

References

- Christ University CIA-3 Project Development Instructions – Advanced JavaScript Backend Frameworks (Node.js & Express JS)
- L&T EduTech Project Specification – Job Portal & Recruitment Management System
- Node.js Documentation
- Express.js Documentation
- MongoDB Documentation
- Mongoose Documentation
- JSON Web Token Documentation
- bcrypt Documentation
- Postman Documentation
- GitHub Documentation

Sensitivity: LNT EduTech Confidential – Christ University 5th Semester Academic Use Only